

HOROSPHERE

Rapport de Tests - Jalon 4

Genere le 24 mai 2026

Informations du projet

Projet :	Horosphere - Gestion RH & pointages geolocali ses
Depot GitHub :	https://github.com/ChahrazedsItn/Horosphere
Branche :	main (dernier commit : 65d5099)
Date du rapport :	24 mai 2026
Framework :	Symfony 7 / PHPUnit 11
Environnement :	GitHub Actions CI - PHP 8.3 + MySQL 8.0

Resume global

Categorie	Fichier	Tests	Resultat
Tests Unitaires	GeofencingServiceTest	10	PASS
Tests Fonctionnels	PointageControllerTest	9	PASS
Tests Fonctionnels	AlerteControllerTest	8	PASS
Tests Fonctionnels	SiteControllerTest	10	PASS
Tests Fonctionnels	DemandeControllerTest	11	PASS
Tests Fonctionnels	UserControllerTest	12	PASS
Tests Fonctionnels	DocumentControllerTest	8	PASS
TOTAL		68	68 / 68

TOUS LES TESTS PASSENT

Total : 68 Reussis : 68 Echoues : 0

1. Introduction

Horosphere est une application web full-stack de gestion des ressources humaines dédiée au suivi des pointages géolocalisés, à la gestion des demandes de congés, aux alertes RH et à l'export de rapports. Elle repose sur une architecture decouplée : backend API REST (Symfony 7 + JWT) et frontend SPA (React + Vite).

Ce rapport couvre l'intégralité de la stratégie de tests mise en place dans le cadre du Jalon 4. Il documente :

- Les tests unitaires de la couche Service (GeofencingService)
- Les tests fonctionnels (HTTP end-to-end) des 6 controllers backend
- Les résultats d'exécution en intégration continue (GitHub Actions)
- La couverture des scénarios de sécurité (contrôle d'accès par rôle)

Objectifs de la stratégie de tests

Fiabilité métier	Vérifier que chaque règle métier (pointage, demande, alerte) se comporte comme attendu.
Sécurité RBAC	S'assurer que les routes protégées renvoient 401/403 pour les rôles non autorisés.
Validation des données	Contrôler que les champs manquants ou invalides retournent 422.
Non-regression	Tout commit sur main déclenche automatiquement la suite de tests via GitHub Actions CI.
Isolation des tests	Chaque test est indépendant (kernel shutdown entre clients, helpers créerDemande).

Architecture technique des tests

Les tests sont organisés en deux catégories dans `api/tests/` :

- `tests/Unit/Service/` Tests unitaires (PHPUnit pur, sans BDD)
- `tests/Functional/Controller/` Tests fonctionnels (WebTestCase, MySQL CI)

Les tests fonctionnels utilisent Symfony WebTestCase qui monte un kernel complet et exécute de vraies requêtes HTTP internes. L'authentification est simulée via l'endpoint `POST /api/auth/login` qui retourne un token JWT réel. Les fixtures (DataFixtures) pré-chargent des utilisateurs avec des rôles AGENT, RH et ADMIN ainsi que des données de demo pour sites, alertes et demandes.

Intégration Continue - GitHub Actions

Un workflow `.github/workflows/ci.yml` déclenche automatiquement les trois jobs suivants à chaque push sur main :

- Backend Tests (PHPUnit) : PHP 8.3, MySQL 8.0, migrations + fixtures
- Frontend Build & Tests : Node 20, Vite, Vitest
- Docker Build : Construction image production via docker buildx

Dernier run CI valide : run #26347540647 [OK] 68/68 tests - 0 échec

Depot : <https://github.com/ChahrazedsItn/Horosphere>

CI : <https://github.com/ChahrazedsItn/Horosphere/actions>

2. Tests Unitaires

GeofencingServiceTest (10 tests)

Le service GeofencingService calcule des distances GPS par la formule de Haversine et determine si un agent se trouve dans le perimetre géographique d'un site. Ces tests sont purement unitaires : SiteRepository est mocké, aucune base de données n'est sollicitée.

[OK]	testCalculerDistanceMemePoint	Distance nulle pour un meme point (delta < 0.01 m)
[OK]	testCalculerDistanceParisMarseilleApprox	Distance Paris->Marseille entre 650 km et 680 km
[OK]	testEstDansZoneVrai	Meme coordonnee que le site -> dans la zone
[OK]	testEstDansZoneFaux	Lyon -> hors zone Paris (rayon 200 m)
[OK]	testEstDansZoneGeofencingDesactive	Geofencing desactive -> toujours dans zone
[OK]	testEstDansZoneFrontiereRayon	Point a ~180 m -> dans zone (rayon 200 m)
[OK]	testDistanceSymetrique	dist(A,B) = dist(B,A) a 0.01 m pres
[OK]	testCalculerDistancePointsProches	~0.001deg lat -> distance entre 50 m et 200 m
[OK]	testValiderCoordonneesValides	Coordonnees valides (+-90/+180) acceptees
[OK]	testValiderCoordonneesInvalides	lat>90, lat<-90, lon>180, lon<-180 rejetees

Resultat

[OK] 10 / 10 tests unitaires reussis

3. Tests Fonctionnels

Les tests fonctionnels couvrent 6 controllers et 58 scenarios. Chaque test effectue une vraie requete HTTP (kernel Symfony complet) et verifie le code HTTP retourne ainsi que la structure JSON de la reponse.

3.1 PointageController - Auth & Pointages (9 tests)

[OK]	testLoginReturnsToken	POST /api/auth/login -> 200 + token JWT
[OK]	testLoginInvalidCredentials	Mauvais mot de passe -> 401 Unauthorized
[OK]	testMeReturnUser	GET /api/auth/me -> profil agent (prenom, role)
[OK]	testArriverRequiresAuth	POST /api/pointages/arriver sans token -> 401
[OK]	testArriverWithValidCoords	Pointage arrivee avec coords valides -> 201
[OK]	testArriverMissingCoords	Corps vide -> 422 Unprocessable Entity
[OK]	testMesPointagesReturnsList	GET /api/pointages/mes-pointages -> liste[]
[OK]	testListePointagesRequiresRh	Agent accede /api/pointages -> 403 Forbidden
[OK]	testListePointagesAsRh	RH accede /api/pointages -> 200

3.2 AlerteController - Alertes (8 tests)

[OK]	testMesAlertesRequiresAuth	GET /api/alertes sans token -> 401
[OK]	testMesAlertesAsAgent	Reponse contient 'data' et 'non_lues'
[OK]	testToutesAlertesRequiresRh	Agent accede /api/alertes/toutes -> 403
[OK]	testToutesAlertesAsRh	RH accede /api/alertes/toutes -> 200
[OK]	testMarquerLueAsOwner	PATCH /lire par le proprietaire -> estLue=true
[OK]	testMarquerLueRefuseAutreUtilisateur	Agent2 marque alerte agent1 -> 403
[OK]	testMarquerToutLu	PATCH /tout-lire -> toutes alertes lues
[OK]	testMarquerToutLuRequiresAuth	PATCH /tout-lire sans token -> 401

3.3 SiteController - Sites geographiques (10 tests)

[OK]	testListeSitesAsAgent	Agent authentifie -> liste >= 3 sites
[OK]	testListeSitesRequiresAuth	Sans token -> 401
[OK]	testDetailSite	GET /api/sites/{id} -> nom correct
[OK]	testCreerSiteRequiresAdmin	Agent cree site -> 403
[OK]	testCreerSiteRhForbidden	RH cree site -> 403
[OK]	testCreerSiteAsAdmin	Admin cree site -> 201 + donnees verifiees
[OK]	testCreerSiteRayonTropPetit	Rayon 5 m -> 422
[OK]	testCreerSiteRayonTropGrand	Rayon 100 000 m -> 422
[OK]	testModifierSiteAsAdmin	PATCH admin -> nom mis a jour
[OK]	testSupprimerSiteAsAdmin	DELETE admin -> 200

3.4 DemandeController - Conges & absences (11 tests)

[OK]	testListeDemandesRequiresAuth	Sans token -> 401
[OK]	testListeDemandesAsAgent	Agent voit uniquement ses demandes
[OK]	testListeDemandesAsRhVoitTout	RH voit >= 3 demandes toutes origines

[OK]	testEnAttenteRequiresRh	Agent accede /en-attente -> 403
[OK]	testEnAttenteAsRh	RH : toutes en statut EN_ATTENTE
[OK]	testCreerDemandeAsAgent	Creation demande -> 201, statut EN_ATTENTE
[OK]	testCreerDemandeChampsManquants	Corps incomplet -> 422
[OK]	testTraiterDemandeApprouveeAsRh	RH approuve -> statut APPROUVEE
[OK]	testTraiterDemandeRejeteeAsRh	RH rejette -> statut REJETEE
[OK]	testTraiterDemandeDecisionInvalide	Decision invalide -> 422
[OK]	testTraiterDemandeRequiresRh	Agent traite -> 403

3.5 UserController - Gestion utilisateurs (12 tests)

[OK]	testListeUsersRequiresAuth	Sans token -> 401
[OK]	testListeUsersAsAgentForbidden	Agent accede /api/users -> 403
[OK]	testListeUsersAsRh	RH -> liste >= 5 utilisateurs
[OK]	testDetailUserAsRh	GET /api/users/{id} -> email et prenom corrects
[OK]	testCreerUserRequiresAdmin	RH cree utilisateur -> 403
[OK]	testCreerUserChampsManquants	Corps incomplet -> 422
[OK]	testCreerUserAsAdmin	Admin cree agent -> 201 + role AGENT
[OK]	testModifierUserAsAdmin	PATCH admin -> departement mis a jour
[OK]	testSupprimerUserAsAdmin	DELETE admin (user temporaire) -> 200
[OK]	testSupprimerPropreCompteForbidden	Admin supprime son propre compte -> 422
[OK]	testStatsDashboardAsAgent	Agent accede dashboard stats -> 403
[OK]	testStatsDashboardAsRh	RH stats : cles total, presents, demandes, taux

3.6 DocumentController - Exports & telechargements (8 tests)

[OK]	testListeDocumentsRequiresAuth	Sans token -> 401
[OK]	testListeDocumentsAsAgent	GET /api/documents -> tableau 'data'
[OK]	testExportCsvRequiresRh	Agent exporte CSV -> 403
[OK]	testExportCsvDatesManquantes	Corps sans dates -> 422
[OK]	testExportCsvAsRh	RH exporte CSV -> 201 + typeDocument=CSV
[OK]	testExportCsvAvecUtilisateurCible	Export CSV pour agent cible -> 201
[OK]	testTelechargerDocumentAsOwner	Telechargement propriétaire -> 200 text/*
[OK]	testTelechargerDocumentAutreUtilisateurForbidden	Agent telecharge doc RH -> 403

4. Resultats d'execution CI

Dernier run GitHub Actions - run #26347540647

Date : 24 mai 2026 - 02:34 UTC
Branche : main (commit 65d5099)
Declencheur : git push - fix(tests): assouplir assertion Content-Type CSV

Job	Resultat	Statut	Duree
Backend Tests (PHPUnit)	68 / 68	SUCCESS	1m 30s
Frontend Build & Tests	Vitest OK	SUCCESS	28s
Docker Build	Image construite	SUCCESS	52s

TOUS LES TESTS PASSENT

Total : 68 Reussis : 68 Echoues : 0

Historique des runs

Run ID	Date	Statut	Note
#26347540647	24 mai 2026 02:34	SUCCESS	Fix Content-Type assertion
#26347508602	24 mai 2026 00:31	FAILURE	Content-Type text/plain vs text/csv
#26347325935	24 mai 2026 00:20	FAILURE	Auth manquante, adresse requise, isolation
#26347130414	24 mai 2026 00:09	SUCCESS	CI stable avant ajout tests controllers
#26346975933	24 mai 2026 00:01	SUCCESS	Fixes securite P0/P1/P2

5. Couverture securite - Controle d'accès RBAC

Une part importante des tests vérifie les règles de contrôle d'accès basé sur les rôles (RBAC). Le tableau ci-dessous recense les 19 tests dédiés à la sécurité (401 / 403 / 422).

Code	Nom du test	Scénario
401 - Non authentifié	testListeDemandesRequiresAuth	Demandes sans token
401 - Non authentifié	testListeSitesRequiresAuth	Sites sans token
401 - Non authentifié	testArriverRequiresAuth	Pointage sans token
401 - Non authentifié	testMesAlertesRequiresAuth	Alertes sans token
401 - Non authentifié	testListeDocumentsRequiresAuth	Documents sans token
401 - Non authentifié	testListeUsersRequiresAuth	Users sans token
401 - Non authentifié	testMarquerToutLuRequiresAuth	Tout-lire sans token
403 - Role insuffisant	testEnAttenteRequiresRh	Agent accède /en-attente
403 - Role insuffisant	testTraiterDemandeRequiresRh	Agent traite une demande
403 - Role insuffisant	testToutesAlertesRequiresRh	Agent accède /toutes alertes
403 - Role insuffisant	testMarquerLueRefuseAutreUtilisateur	Agent lit alerte d'un autre
403 - Role insuffisant	testCreerSiteRequiresAdmin	Agent crée un site
403 - Role insuffisant	testCreerSiteRhForbidden	RH crée un site
403 - Role insuffisant	testExportCsvRequiresRh	Agent exporte CSV
403 - Role insuffisant	testListeUsersAsAgentForbidden	Agent accède /users
403 - Role insuffisant	testCreerUserRequiresAdmin	RH crée un user
403 - Role insuffisant	testStatsDashboardAsAgent	Agent accède dashboard stats
403 - Role insuffisant	testTelechargerDocumentAutreUtilisateurForbidden	Agent télécharge doc RH
422 - Auto-suppression	testSupprimerPropreCompteForbidden	Admin supprime son propre compte

[OK] 19 / 19 tests de sécurité réussis - aucune régression RBAC détectée.

6. Conclusion

La suite de tests du Jalon 4 couvre l'ensemble des fonctionnalités critiques de l'API Horosphere : authentification JWT, pointages géolocalisés, gestion des demandes de congés, alertes RH, gestion des utilisateurs, sites géographiques et exports de documents.

- 68 tests au total	10 unitaires + 58 fonctionnels, tous verts en CI.
- Couverture RBAC complète	19 tests dédiés à la sécurité (401, 403, 422) pour les 3 rôles.
- Pipeline CI automatisé	Chaque push sur main déclenche la suite complète via GitHub Actions.
- Isolation garantie	ensureKernelShutdown() entre chaque client, helpers creerDemande() pour éviter les dépendances entre tests.
- Zero dette de test	Tous les contrôleurs ont leur test fonctionnel ; GeofencingService a ses tests unitaires Haversine.

Lien de livraison

Depot GitHub : <https://github.com/ChahrazedsItn/Horosphere>

CI / Actions : <https://github.com/ChahrazedsItn/Horosphere/actions>

Branche livrée : main - commit 65d5099